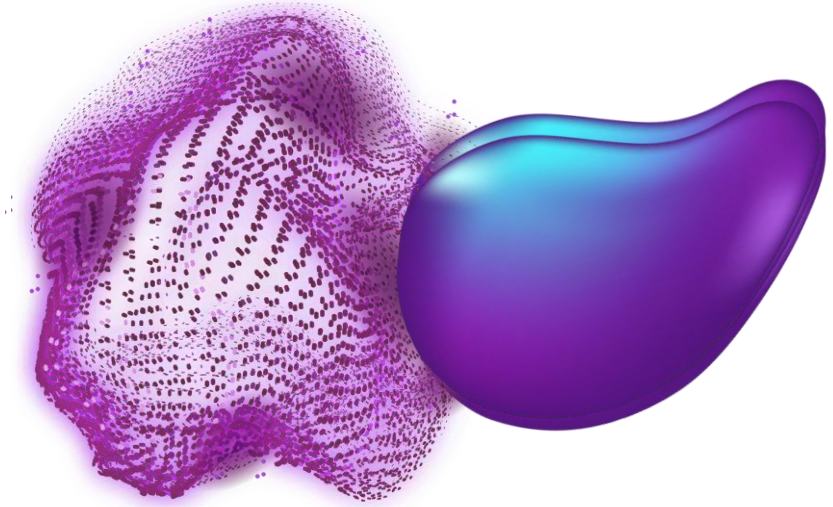


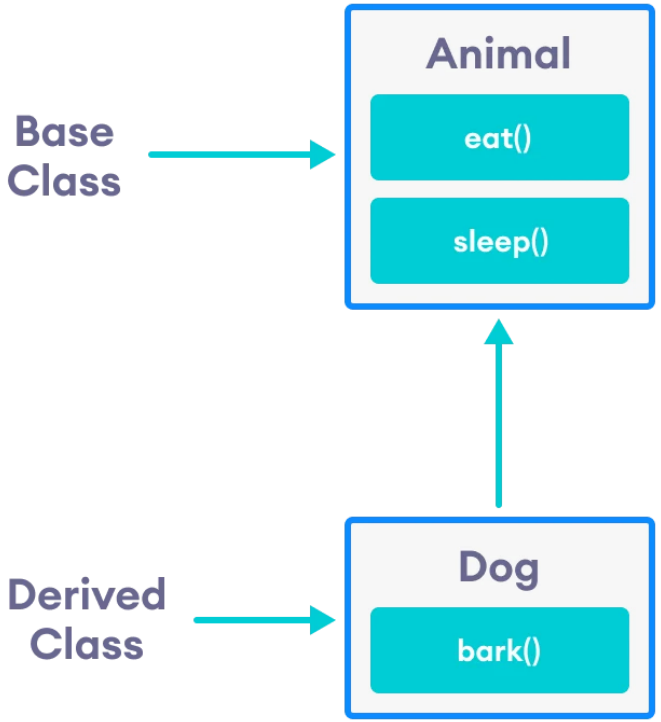
Object Oriented Programming

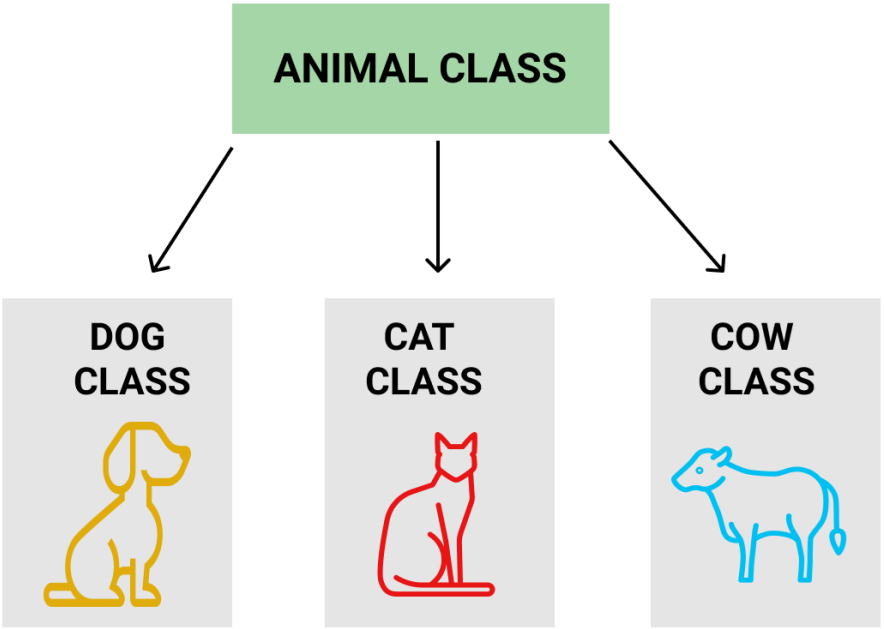
Lecture 8

Inheritance_P1

Inheritance:

- ✓**Inheritance** is the process by which objects of one class acquire the properties of another class.
- ✓In the concept of inheritance provides the idea of **reusability**.
- ✓This mean that we **can add additional features** to an existing class without modifying it.
- ✓ This is possible by designing a new class will have the combined features of both the classes.





Inheritance:

is-a relationship: A hierarchical connection where one category can be treated as a *specialized version* of another.

- every rectangle *is a* shape ** every lion *is an* animal ** every lawyer *is an* employee

inheritance: A way to form new classes based on existing classes, taking on their attributes/behavior.

- a way to group related classes
- a way to share code between two or more classes.

One class can *extend* another, absorbing its data/behavior, The new class is a specialized version of the existing class

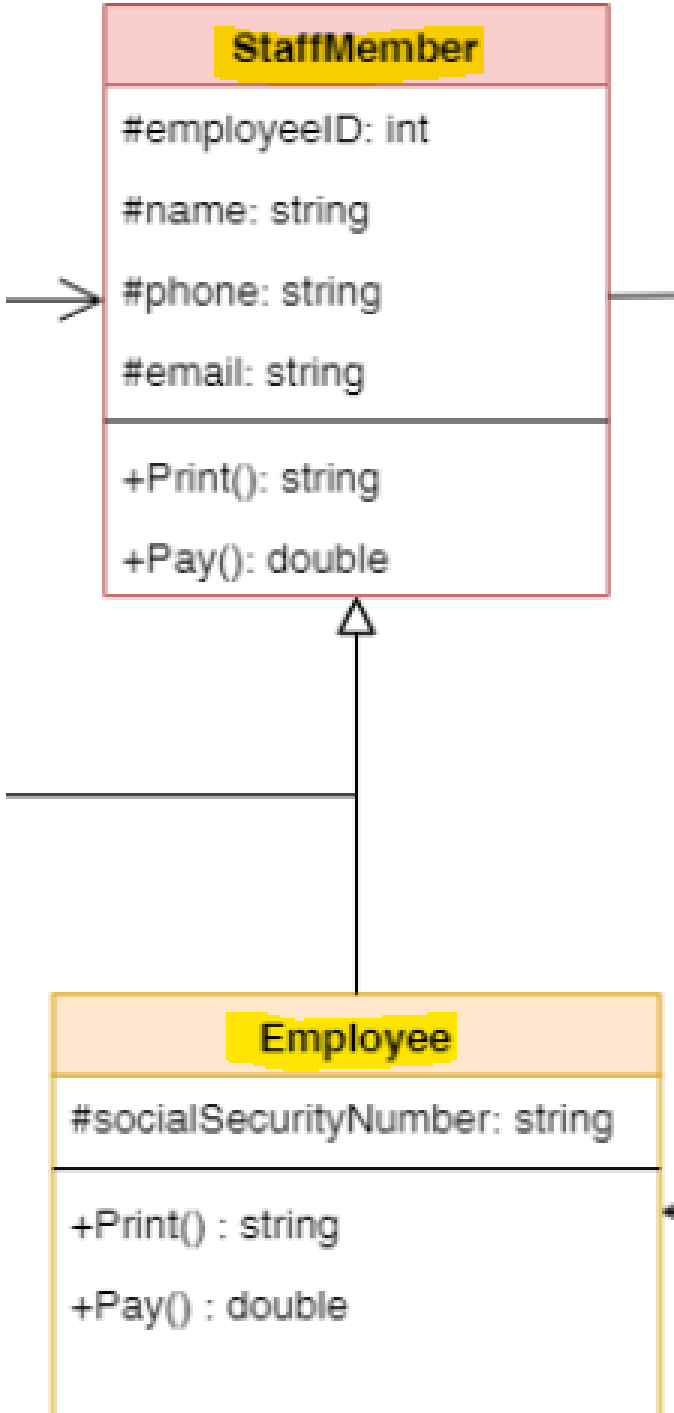
type hierarchy: A set of data types connected by **is-a** relationships that **can share common code**.

Inheritance:

- **superclass** (base class): Parent class that is being **extended**.
- **subclass** (derived class): Child class that **inherits** from the superclass.
  - Subclass gets a copy of every field and method from superclass.
  - Subclass can add its own behavior, and/or change inherited behavior.

Note:-

The **direction** of the **arrow** emphasizes that the **derived class** refers to functions and data in the **base class**, while the base class has no access to the derived class.

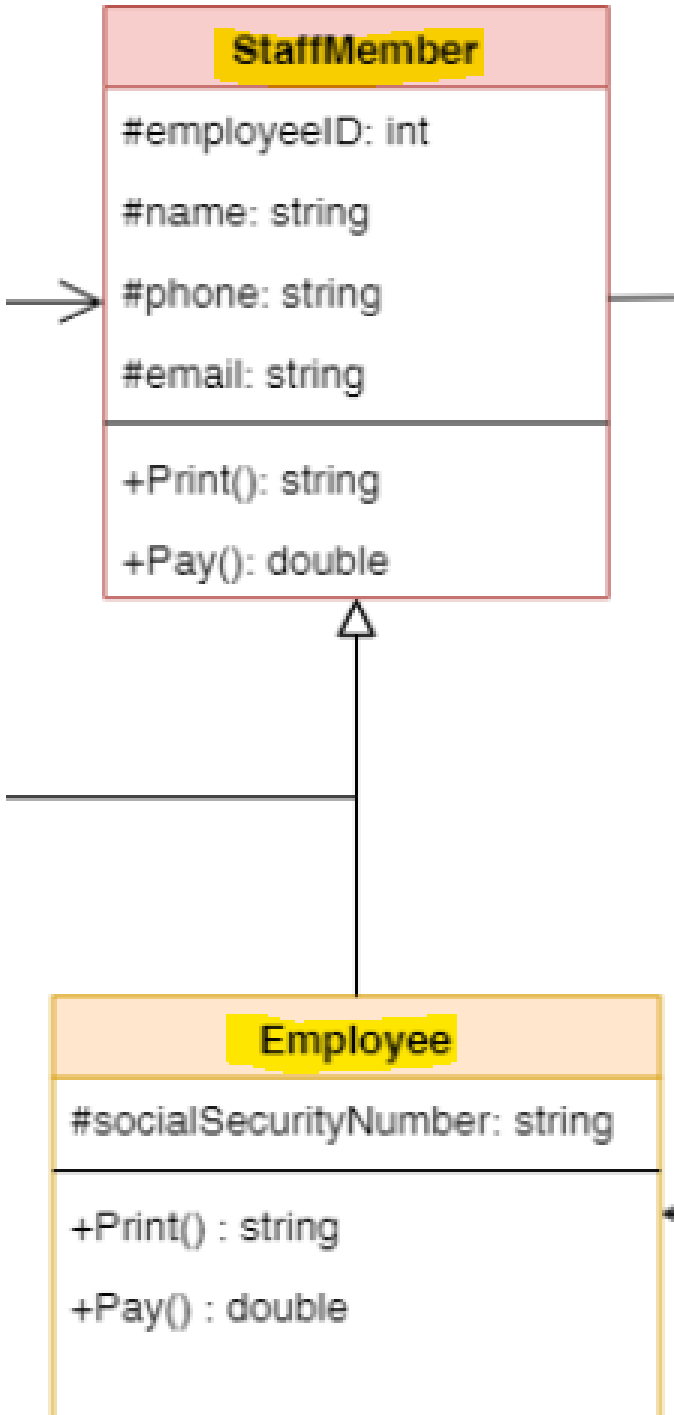


```cpp
class staffmember{
protected:
    int empolyeeid;
    string name,phone,email;
public:
    void print();
    double pay();
};
class empolyee:staffmember{};
```

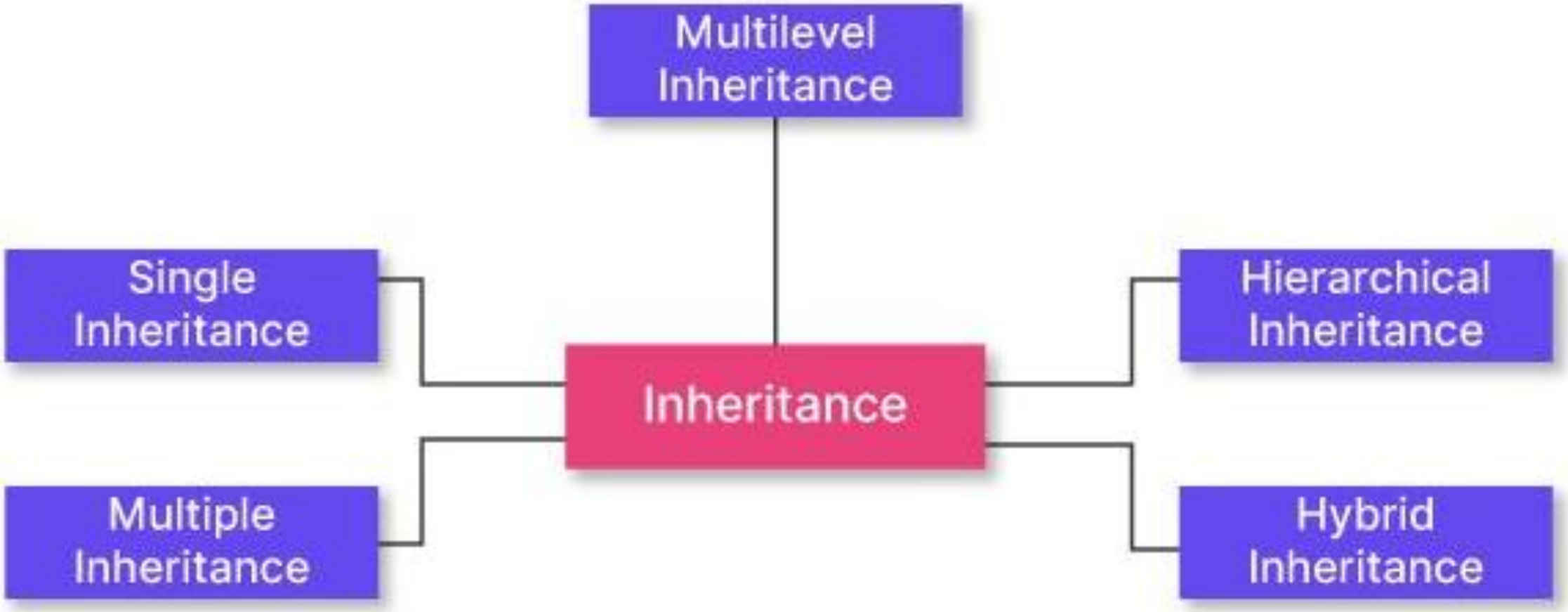
```cpp
int main(){
    empolyee emp;
    emp.
}
```

email
empolyeeid
name
pay
phone
print

protected : std::string staffmember::email
File: Source.cpp

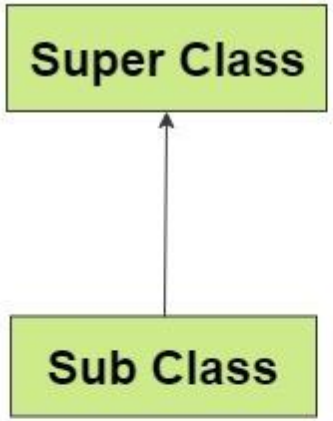


Types of inheritance

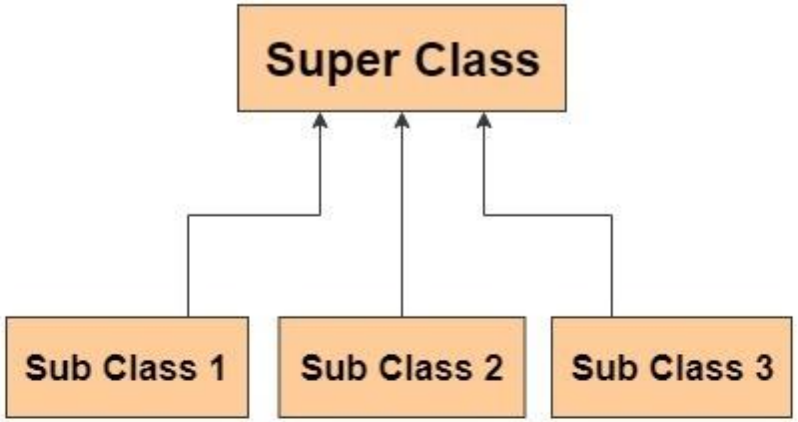


Inheritance Types:

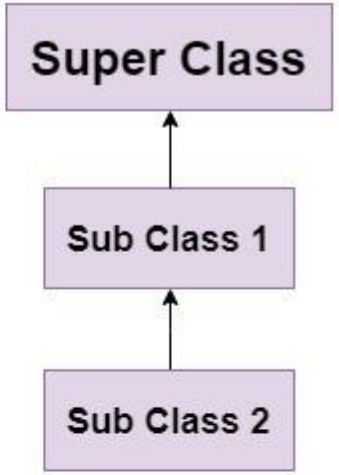
Single Inheritance



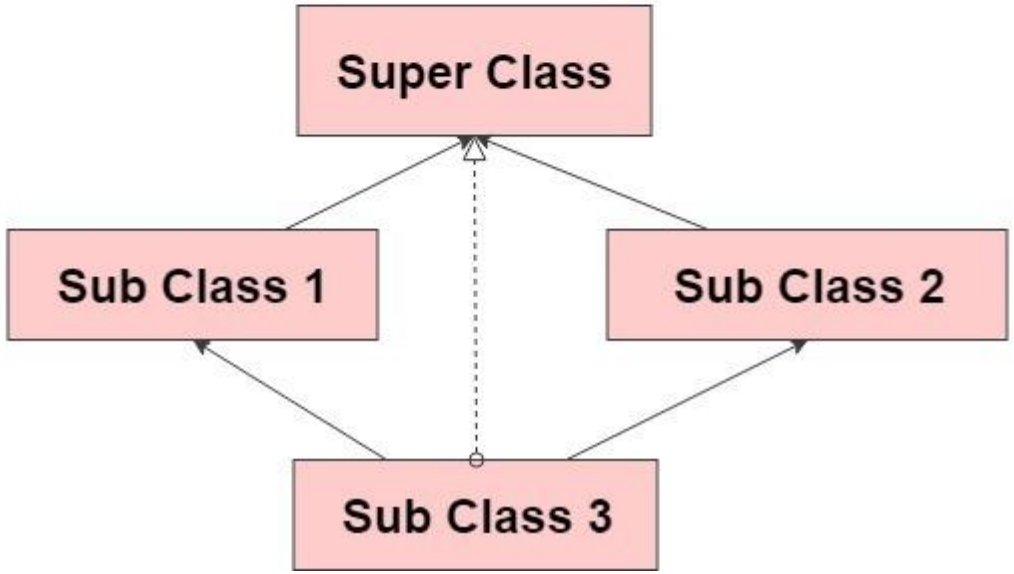
Hierarchial Inheritance



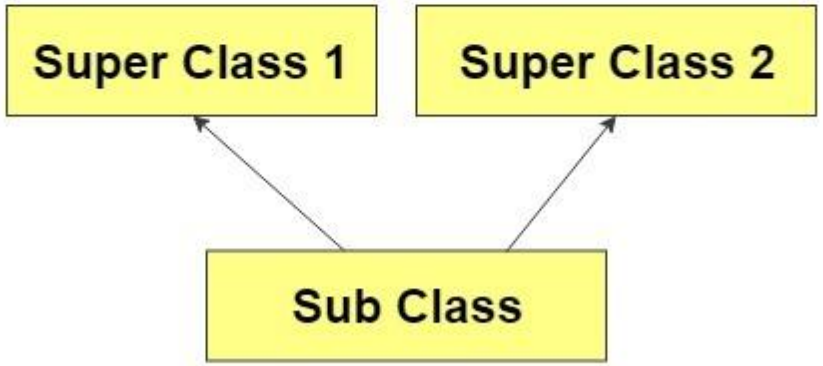
MultiLevel Inheritance



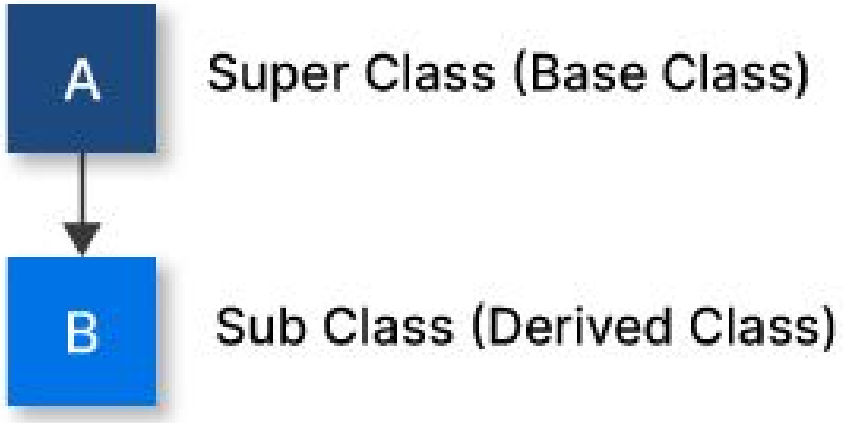
Hybrid Inheritance



Multiple Inhertance

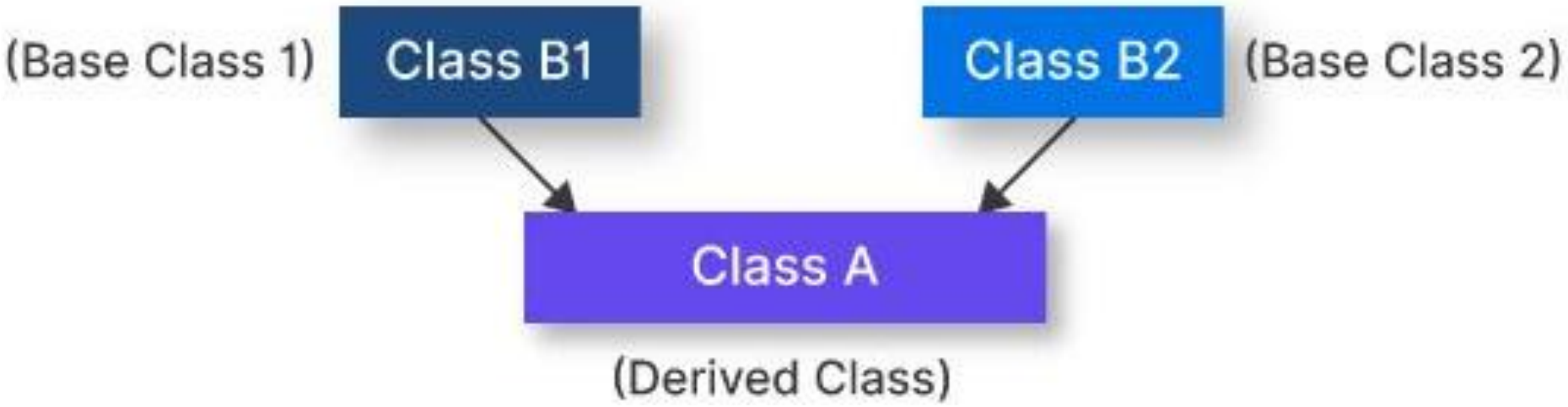


Single Inheritance




```
class Baseclass{

};
class Derivedclass:private Baseclass{

};
```

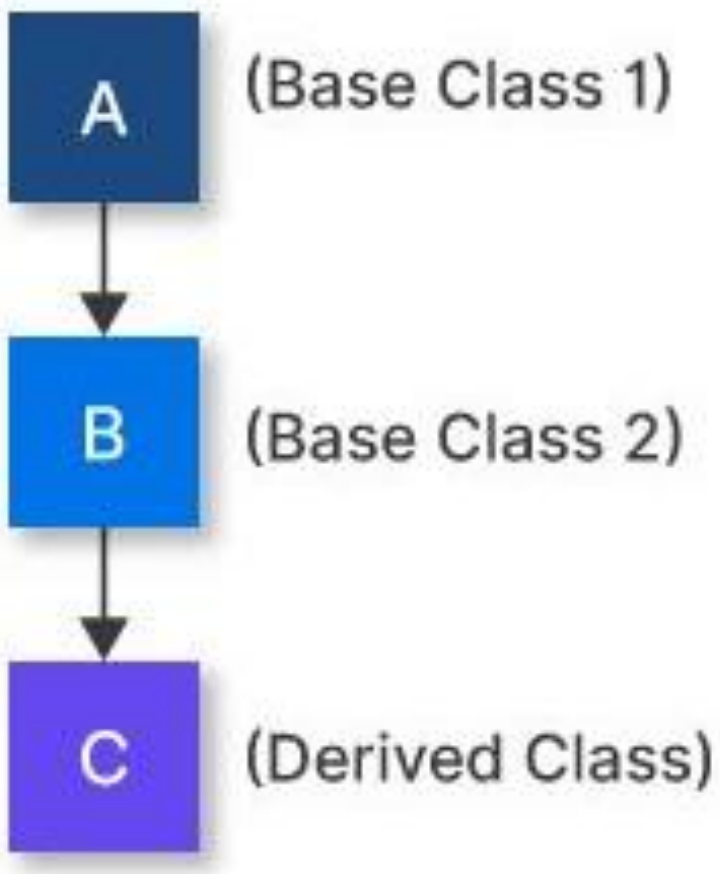
Multiple Inheritance



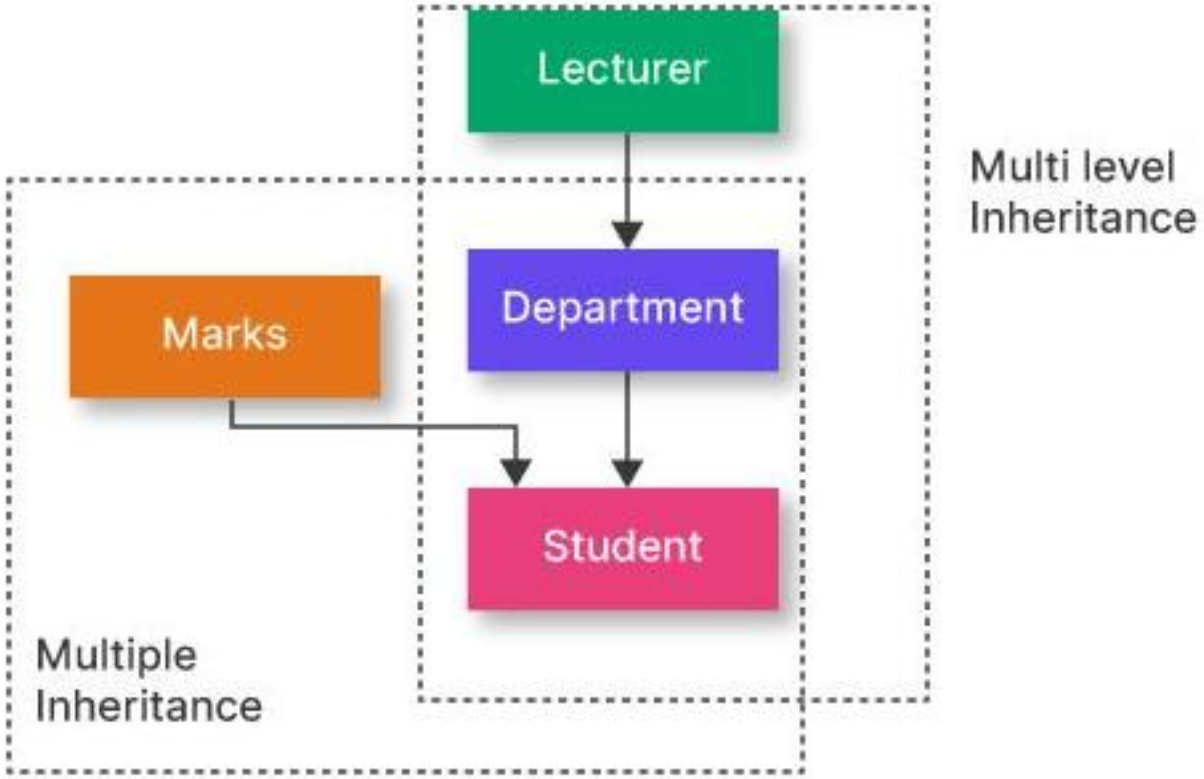

```
class Baseclass1{

};
class Baseclass2{

};
class Derivedclass: private Baseclass1, public Baseclass2{

};
```

Multi-level Inheritance

```
class Baseclass1{

};
class Baseclass2: public Baseclass1{

};
class Derivedclass: protected Baseclass2 {

};
```



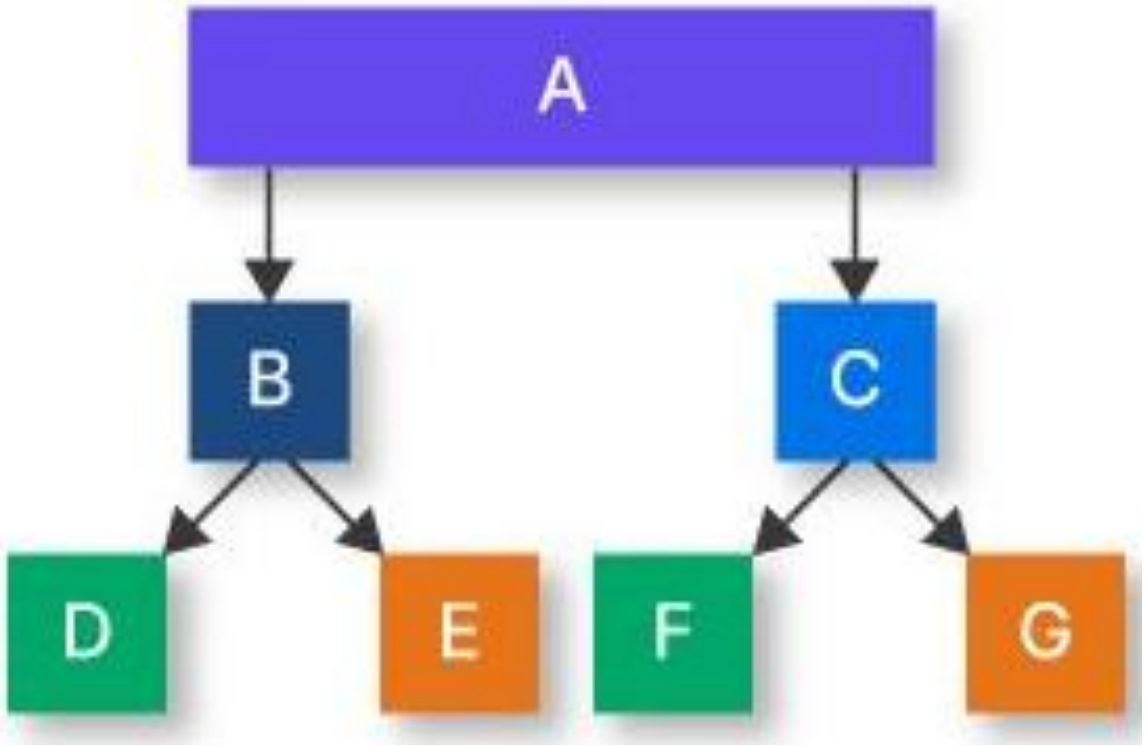
Hybrid Inheritance




```
class lecture{

};
class Department: public lecture{

};
class Marks{

};
class Student: protected Department, public Marks {

};
```

Hierarchal Inheritance

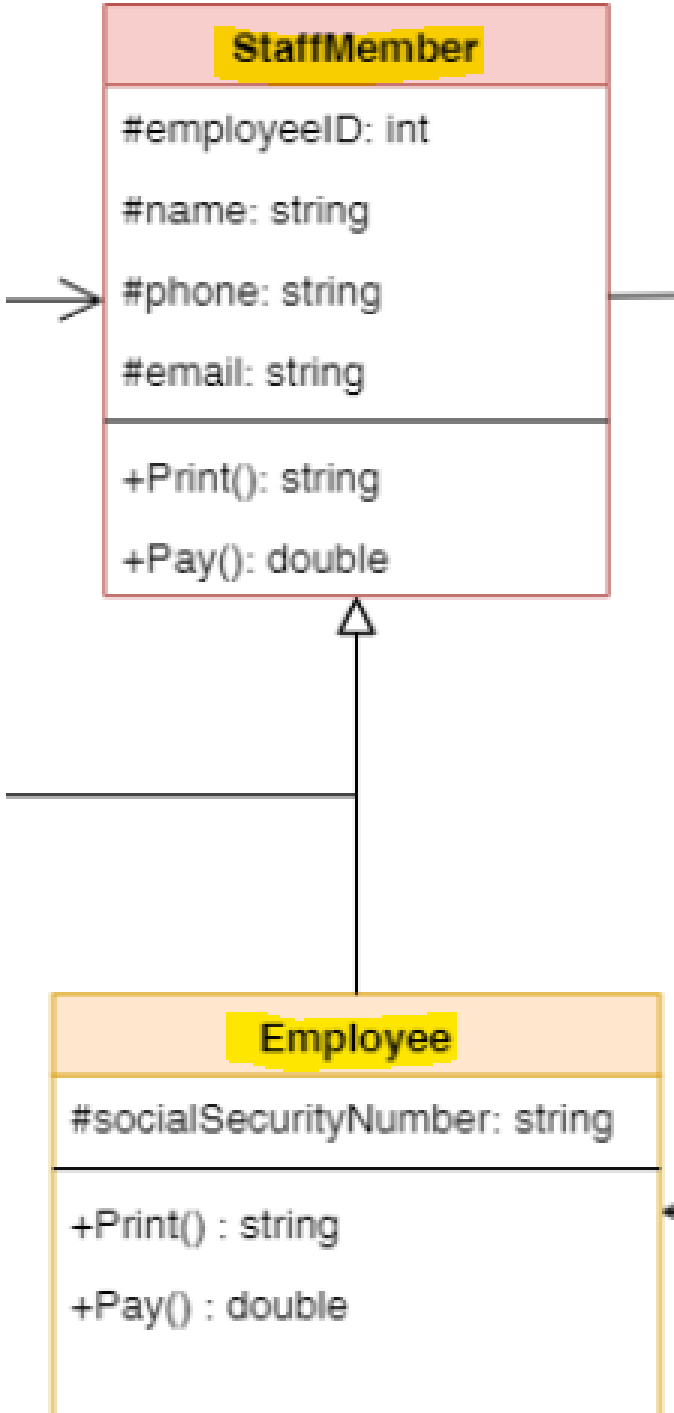
```
class Shape{

};
class Rectangle: public Shape{

};
class circle: private Shape{

};
class triangle: protected Shape{

};
```



```
class empolyee:staffmember{
protected:
    int socialsecuritynumber;
public:
    void print();
    double pay();
};
```

```
empolyee emp;
emp.
```

- email
- empolyeeid
- name
- pay
- phone
- print
- socialsecuritynumber



An *object* of the *derived class* has:

- all members defined in child class
- all members declared in parent class

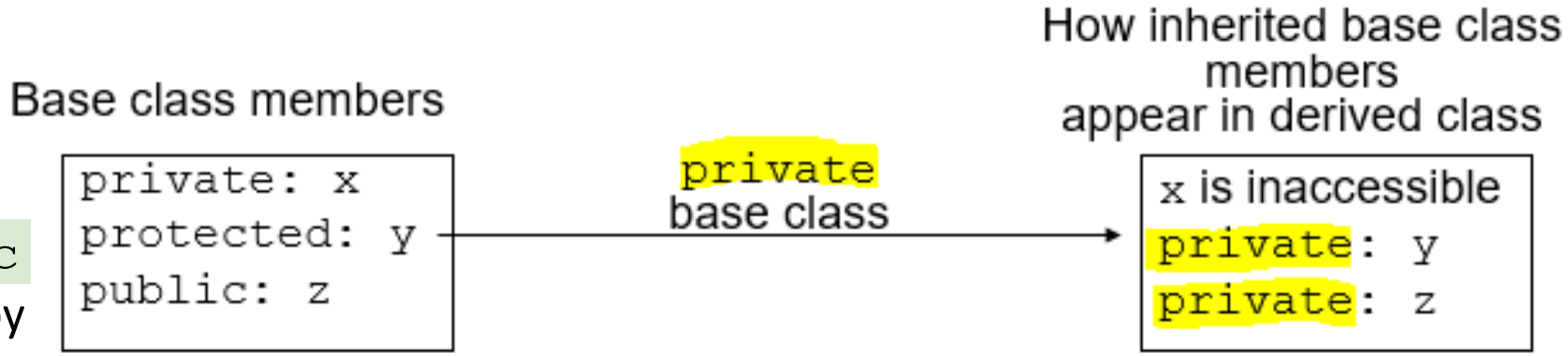
An **object** of the *derived class* can use:

- all `public` members defined in child class
- all `public` members defined in parent class

Class access specification

Determines how

`private`, `protected`, and `public` members of **base class** are **inherited** by the **derived class**

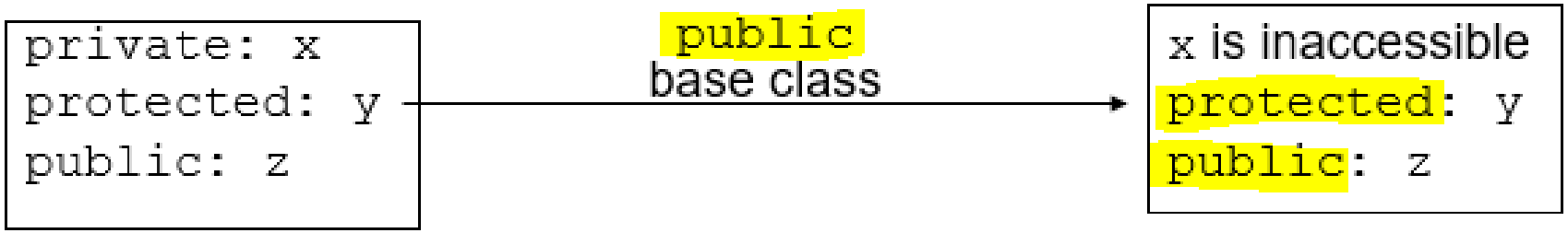



```
class A //base class
{
private:
    int privdataA;
protected:
    int protdataA;
public:
    int pubdataA;
};
```

```
class C : private A //privately-derived class
{
public:
    void funct()
    {
        int a;
        a = privdataA; //error: not accessible
        a = protdataA; //OK
        a = pubdataA; //OK
    }
};
```

```
C objC;
a = objC.privdataA; //error: not accessible
a = objC.protdataA; //error: not accessible
a = objC.pubdataA; //error: not accessible (A private to C)
```

Class access specification

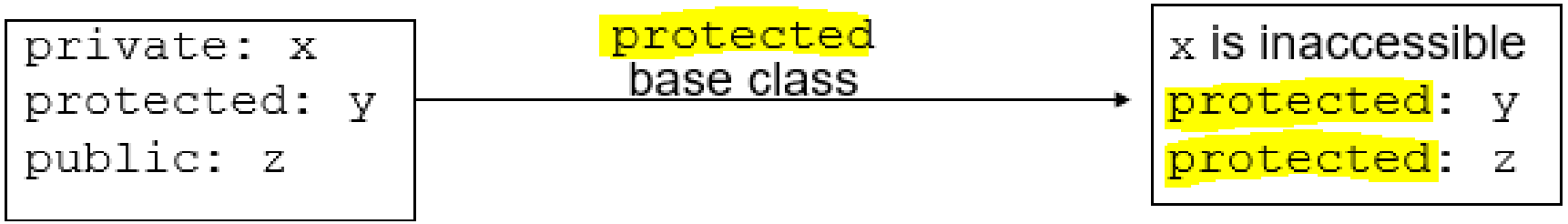



```
class A //base class
{
private:
    int privdataA;
protected:
    int protdataA;
public:
    int pubdataA;
};
```

```
class B : public A //publicly-derived class
{
public:
    void funct()
    {
        int a;
        a = privdataA; //error: not accessible
        a = protdataA; //OK
        a = pubdataA; //OK
    }
};
```

```
B objB;
a = objB.privdataA; //error: not accessible
a = objB.protdataA; //error: not accessible
a = objB.pubdataA; //OK (A public to B)
```

Class access specification

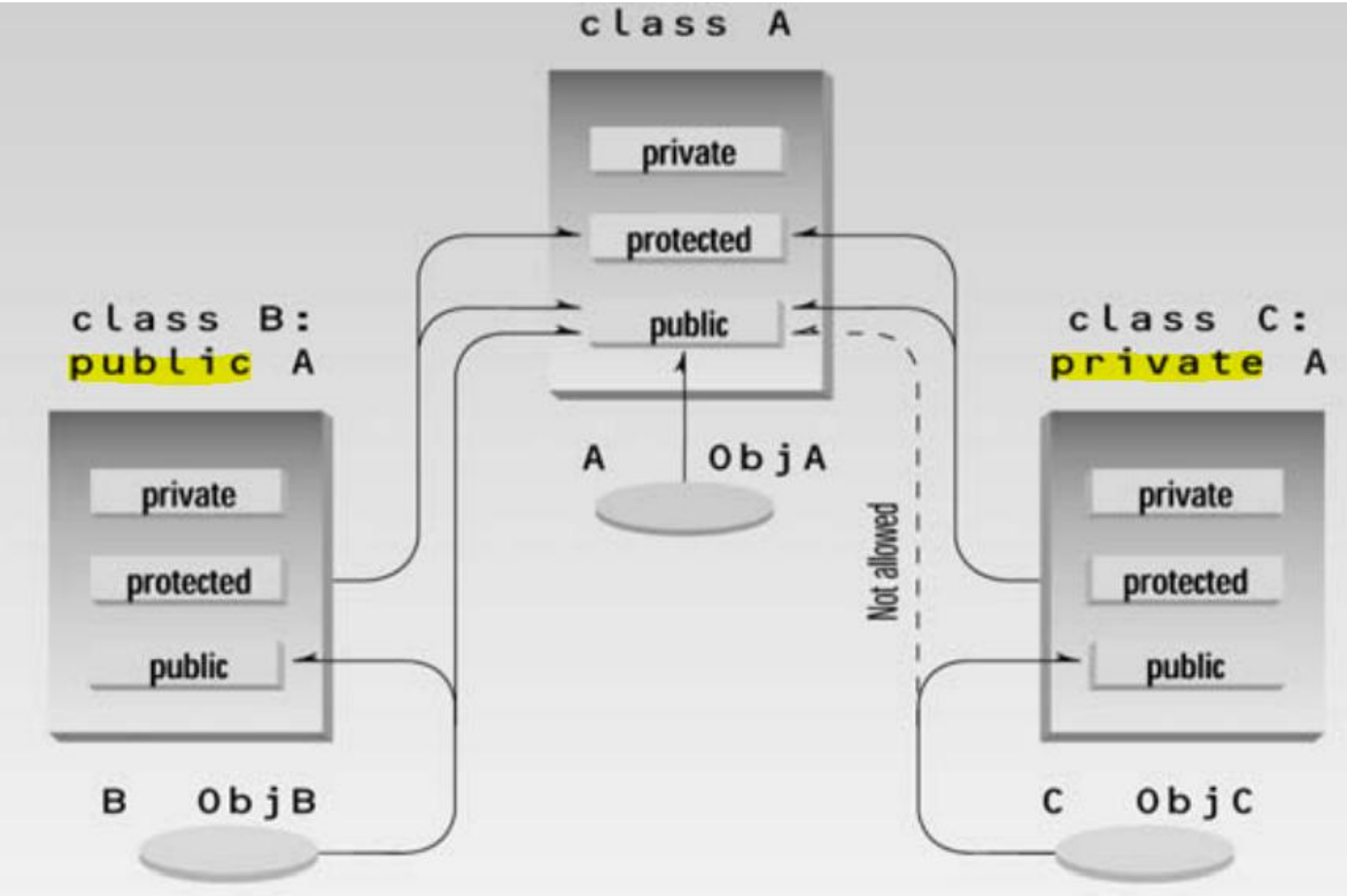



```
class A //base class
{
private:
    int privdataA;
protected:
    int protdataA;
public:
    int pubdataA;
};
```

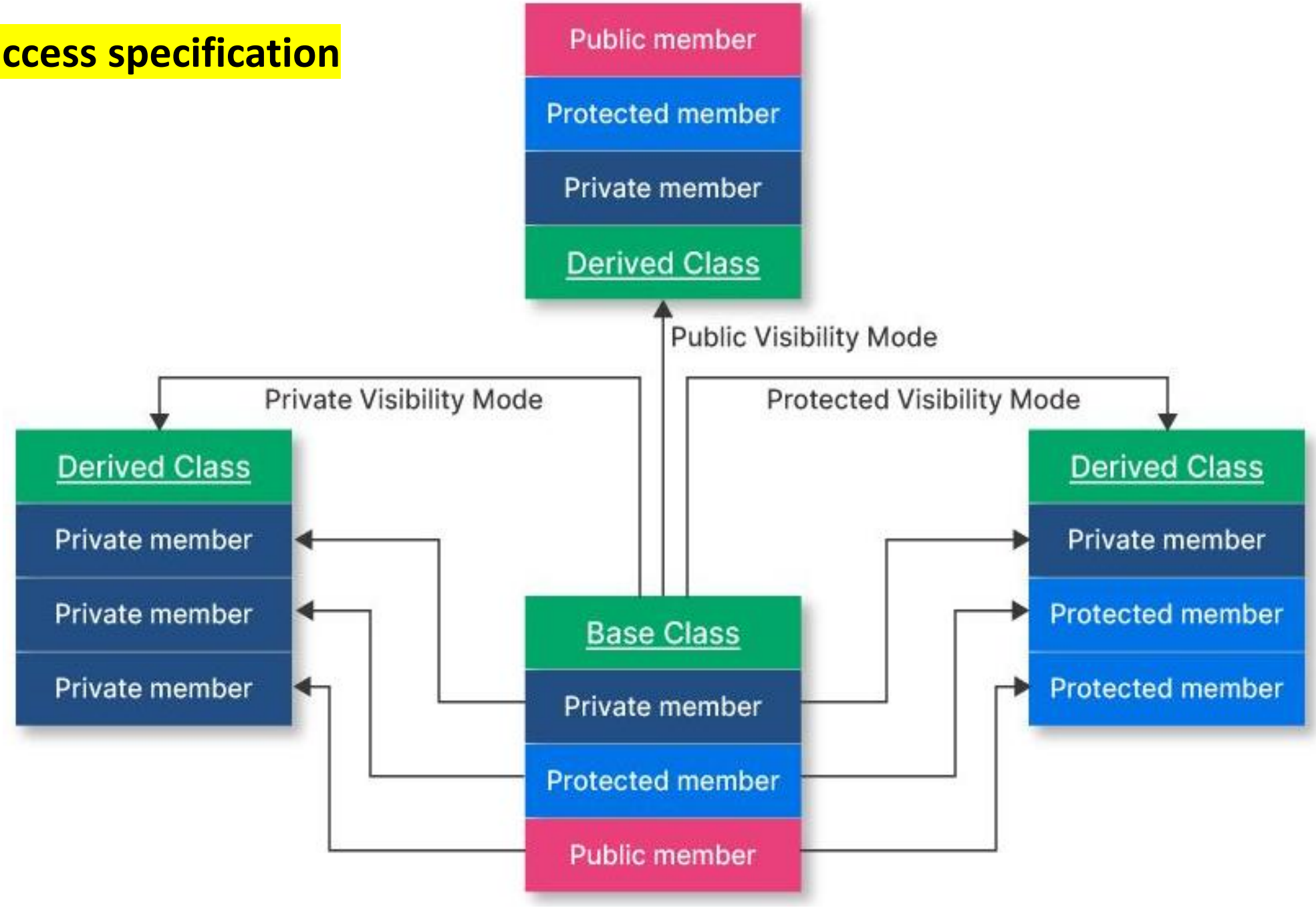
```
class B : protected A //protected-derived class
{
public:
    void funct()
    {
        int a;
        a = privdataA; //error: not accessible
        a = protdataA; //OK
        a = pubdataA; //OK
    }
};
```

```
B objB;
a = objB.privdataA; //error: not accessible
a = objB.protdataA; //error: not accessible
a = objB.pubdataA; //error: not accessible
```

Class access specification



Class access specification



Class access specification

Base Class Visibility	Derived Class Visibility		
	Public	Protected	Private
Public	Public	Protected	Private
Protected	Protected	Protected	Private
Private	Not accessible (Hidden)	Not accessible (Hidden)	Not accessible (Hidden)

Constructors and Destructors in Base and Derived Classes

Order of Inheritance

Class C (Base Class 2)
↓
Class B (Base Class 1)
↓
Class A (Derived Class)

- ✓ Derived classes can have their own constructors and destructors.
- ✓ When an object of a derived class is created, the base class's constructor is executed first, followed by the derived class's constructor
- ✓ When an object of a derived class is destroyed, its destructor is called first, then that of the base class

Order of Constructor Call

1. C() (Class C's Constructor)
2. B() (Class B's Constructor)
3. A() (Class A's Constructor)

Order of Destructor Call

1. ~A() (Class A's Destructor)
2. ~B() (Class B's Destructor)
3. ~C() (Class C's Destructor)

```
class baseclass{
public:
    baseclass(){
        cout<<"base class Constructor"<<endl;
    }
    ~baseclass(){
        cout<<"base class Destructor"<<endl;
    }
};

class childclass:baseclass{
public:
    childclass(){
        cout<<"Child class Constructor"<<endl;
    }
    ~childclass(){
        cout<<"Child class Destructor"<<endl;
    }
};
```

```
int main(){
    {
        childclass ch;
    }
    system("Pause");
}
```

```
base class Constructor
Child class Constructor
Child class Destructor
base class Destructor
Press any key to continue . . .
```

Calling constructor in base class & passing argument to it

derived class constructor

base class constructor

```
Square::Square(int side):Rectangle(side,side)
```

derived constructor parameter

base constructor parameters

```
class baseclass{
protected:
    int id;
    string name,add;
public:
    baseclass(int _id,string _name,string _add){
        id=_id;
        name=_name;
        add=_add;
    }

  //calling base class's constructor using colon :
  childclass(int id,string n,string a):baseclass(id,n,a){
      cout<<"child parameter cons. \n";
  }
```

```
childclass ch(5,"aly","zag");
```

Multiple Inheritance Problem

- ✓ A derived class can have more than one base class
- ✓ Each base class can have its own access specification in derived class's definition:

What is the problem?

- ✓ What if base classes have member variables/functions with the same name?
- ✓ Compiler errors occur if derived class uses base class function

Solution ?

- ✓ Derived class ***redefines*** the multiply-defined function
- ✓ Derived class ***invokes*** member function in a particular base class using scope resolution operator **::**

```
class C:public A,public B{
public:
    //Redefine function
    void get();
};
```

```
class C:public A,public B{
public:
    void get(){
        A::get();
    }
};
```

```
class A{
public:
    void get();
    void set();
};
class B{
public:
    void get();
    void set();
};
class C:public A,public B{

};
int main(){
    C cobject;
    cobject.get();

    system("
```

void A::get()

Error: "C::get" is ambiguous

```cpp
class Base {
    public:
      void print() {
          // code
      }
};

class Derived : public Base {
    public:
      void print() {
          // code
      }
};

int main() {
    Derived derived1;
    derived1.print();
    return 0;
}
```

```cpp
class Base {
    public:
      void print() {
          // code
      }
};

class Derived : public Base {
    public:
      void print() {
          // code
      }
};

int main() {
    Derived derived1, derived2;

    derived1.print();

    derived2.Base::print();

    return 0;
}
```

```
class Base {
    public:
      void print() {
          // code
      }
};

class Derived : public Base {
    public:
      void print() {
          // code
          Base::print();
      }
};

int main() {
    Derived derived1;
    derived1.print();
    return 0;
}
```

```
class empolyee{
public:
    void getdata(){
        cout<<"Empolyee (SuperClass) \n";
    }
};
class staff{
public:
    //Override not overload
    // With overloading, parameter lists must be different
    void getdata(){
        cout<<"Staff (SubClass) \n";
    }
};
int main(){
    empolyee p;
    //p use base class version of function;
    p.getdata();
    staff s;
    //s use derived class version of function
    s.getdata();
```

```
Empolyee (SuperClass)
Staff (SubClass)
```